



Small. Fast. Reliable.
Choose any three.

- [Home](#)
 - [Menu](#)
 - [About](#)
 - [Documentation](#)
 - [Download](#)
 - [License](#)
 - [Support](#)
 - [Purchase](#)
-
- [About](#)
 - [Documentation](#)
 - [Download](#)
 - [Support](#)
 - [Purchase](#)

Search Documentation ▼

Go

Isolation In SQLite

The "isolation" property of a database determines when changes made to the database by one operation become visible to other concurrent operations.

Isolation Between Database Connections

If the same database is being read and written using two different [database connections](#) (two different [sqlite3](#) objects returned by separate calls to [sqlite3_open\(\)](#)) and the two database connections do not have a [shared cache](#), then the reader is only able to see complete committed transactions from the writer. Partial changes by the writer that have not been committed are invisible to the reader. This is true regardless of whether the two database connections are in the same thread, in different threads of the same process, or in different processes. This is the usual and expected behavior for SQL database systems.

The previous paragraph is also true (separate database connections are isolated from one another) in [shared cache mode](#) as long as the [read_uncommitted pragma](#) remains turned off. The [read_uncommitted pragma](#) is off by default and so if the application does nothing to turn it on, it will remain off. Hence, unless the [read_uncommitted pragma](#) is used to change the default behavior, changes made by one database connection are invisible to readers on a different database connection sharing the same cache until the writer commits its transaction.

If two database connections share the same cache and the reader has enabled the [read_uncommitted pragma](#), then the reader will be able to see changes made by the writer before the writer transaction commits. The combined use of [shared cache mode](#) and the [read_uncommitted pragma](#) is the only way that one database connection can see uncommitted changes on a different database connection. In all other circumstances, separate database connections are completely isolated from one another.

Except in the case of [shared cache](#) database connections with [PRAGMA read_uncommitted](#) turned on, all transactions in SQLite show "serializable" isolation. SQLite implements serializable transactions by actually serializing the writes. There can only be a single writer at a time to an SQLite database. There can be multiple database connections open at the same time, and all of those database connections can write to the database file,

but they have to take turns. SQLite uses locks to serialize the writes automatically; this is not something that the applications using SQLite need to worry about.

Isolation And Concurrency

SQLite implements isolation and concurrency control (and atomicity) using transient journal files that appear in the same directory as the database file. There are two major "journal modes". The older "rollback mode" corresponds to using the "DELETE", "PERSIST", or "TRUNCATE" options to the [journal_mode pragma](#). In rollback mode, changes are written directly into the database file, while simultaneously a separate rollback journal file is constructed that is able to restore the database to its original state if the transaction rolls back. Rollback mode (specifically DELETE mode, meaning that the rollback journal is deleted from disk at the conclusion of each transaction) is the current default behavior.

Since [version 3.7.0](#) (2010-07-21), SQLite also supports "[WAL mode](#)". In WAL mode, changes are not written to the original database file. Instead, changes go into a separate "write-ahead log" or "WAL" file. Later, after the transaction commits, those changes will be moved from the WAL file back into the original database in an operation called "checkpoint". WAL mode is enabled by running "[PRAGMA journal_mode=WAL](#)".

In rollback mode, SQLite implements isolation by locking the database file and preventing any reads by other database connections while each write transaction is underway. Readers can be active at the beginning of a write, before any content is flushed to disk and while all changes are still held in the writer's private memory space. But before any changes are made to the database file on disk, all readers must be (temporarily) expelled in order to give the writer exclusive access to the database file. Hence, readers are prohibited from seeing incomplete transactions by virtue of being locked out of the database while the transaction is being written to disk. Only after the transaction is completely written and synced to disk and committed are the readers allowed back into the database. Hence readers never get a chance to see partially written changes.

WAL mode permits simultaneous readers and writers. It can do this because changes do not overwrite the original database file, but rather go into the separate write-ahead log file. That means that readers can continue to read the old, original, unaltered content from the original database file at the same time that the writer is appending to the write-ahead log. In [WAL mode](#), SQLite exhibits "snapshot isolation". When a read transaction starts, that reader continues to see an unchanging "snapshot" of the database file as it existed at the moment in time when the read transaction started. Any write transactions that commit while the read transaction is active are still invisible to the read transaction, because the reader is seeing a snapshot of database file from a prior moment in time.

An example: Suppose there are two database connections X and Y. X starts a read transaction using [BEGIN](#) followed by one or more [SELECT](#) statements. Then Y comes along and runs an [UPDATE](#) statement to modify the database. X can subsequently do a [SELECT](#) against the records that Y modified but X will see the older unmodified entries because Y's changes are all invisible to X while X is holding a read transaction. If X wants to see the changes that Y made, then X must end its read transaction and start a new one (by running [COMMIT](#) followed by another [BEGIN](#).)

Another example: X starts a read transaction using [BEGIN](#) and [SELECT](#), then Y makes a changes to the database using [UPDATE](#). Then X tries to make a change to the database using [UPDATE](#). The attempt by X to escalate its transaction from a read transaction to a write transaction fails with an [SQLITE_BUSY_SNAPSHOT](#) error because the snapshot of the database being viewed by X is no longer the latest version of the database. If X were allowed to write, it would fork the history of the database file, which is something SQLite does not support. In order for X to write to the database, it must first release its snapshot (using [ROLLBACK](#) for example) then start a new transaction with a subsequent [BEGIN](#).

If X starts a transaction that will initially only read but X knows it will eventually want to write and does not want to be troubled with possible `SQLITE_BUSY_SNAPSHOT` errors that arise because another connection jumped ahead of it in line, then X can issue [BEGIN IMMEDIATE](#) to start its transaction instead of just an

ordinary BEGIN. The [BEGIN IMMEDIATE](#) command goes ahead and starts a write transaction, and thus blocks all other writers. If the [BEGIN IMMEDIATE](#) operation succeeds, then no subsequent operations in that transaction will ever fail with an [SQLITE_BUSY](#) error.

No Isolation Between Operations On The Same Database Connection

SQLite provides isolation between operations in separate database connections. However, there is no isolation between operations that occur within the same database connection.

In other words, if X begins a write transaction using [BEGIN IMMEDIATE](#) then issues one or more [UPDATE](#), [DELETE](#), and/or [INSERT](#) statements, then those changes are visible to subsequent [SELECT](#) statements that are evaluated in database connection X. [SELECT](#) statements on a different database connection Y will show no changes until the X transaction commits. But [SELECT](#) statements in X will show the changes prior to the commit.

Within a single database connection X, a SELECT statement always sees all changes to the database that are completed prior to the start of the SELECT statement, whether committed or uncommitted. And the SELECT statement obviously does not see any changes that occur after the SELECT statement completes. But what about changes that occur while the SELECT statement is running? What if a SELECT statement is started and the [sqlite3_step\(\)](#) interface steps through roughly half of its output, then some [UPDATE](#) statements are run by the application that modify the table that the SELECT statement is reading, then more calls to [sqlite3_step\(\)](#) are made to finish out the SELECT statement? Will the later steps of the SELECT statement see the changes made by the UPDATE or not? The answer is that this behavior is undefined. In particular, whether or not the SELECT statement sees the concurrent changes depends on which release of SQLite is running, the schema of the database file, whether or not [ANALYZE](#) has been run, and the details of the query. In some cases, it might depend on the content of the database file, too. There is no good way to know whether or not a SELECT statement will see changes that were made to the database by the same database connection after the SELECT statement was started. And hence, developers should diligently avoid writing applications that make assumptions about what will occur in that circumstance.

If an application issues a SELECT statement on a single table like *"SELECT rowid, * FROM table WHERE ..."* and starts stepping through the output of that statement using [sqlite3_step\(\)](#) and examining each row, then it is safe for the application to delete the current row or any prior row using *"DELETE FROM table WHERE rowid=?"*. It is also safe (in the sense that it will not harm the database) for the application to delete a row that expected to appear later in the query but has not appeared yet. If a future row is deleted, however, it might happen that the row turns up after a subsequent [sqlite3_step\(\)](#), even after it has allegedly been deleted. Or it might not. That behavior is undefined. The application can also INSERT new rows into the table while the SELECT statement is running, but whether or not the new rows appear in subsequent [sqlite3_step\(\)](#)s of the query is undefined. And the application can UPDATE the current row or any prior row, though doing so might cause that row to reappear in a subsequent [sqlite3_step\(\)](#). As long as the application is prepared to deal with these ambiguities, the operations themselves are safe and will not harm the database file.

For the purposes of the previous two paragraphs, two database connections that have the same [shared cache](#) and which have enabled [PRAGMA read_uncommitted](#) are considered to be the same database connection.

Summary

1. Transactions in SQLite are **SERIALIZABLE**.
2. Changes made in one database connection are invisible to all other database connections prior to commit.
3. A query sees all changes that are completed on the same database connection prior to the start of the query, regardless of whether or not those changes have been committed.

4. If changes occur on the same database connection after a query starts running but before the query completes, then it is undefined whether or not the query will see those changes.
5. If changes occur on the same database connection after a query starts running but before the query completes, then the query might return a changed row more than once, or it might return a row that was previously deleted.
6. For the purposes of the previous four items, two database connections that use the same [shared cache](#) and which enable [PRAGMA read_uncommitted](#) are considered to be the same database connection, not separate database connections.

This page was last updated on 2022-04-18 02:55:50Z